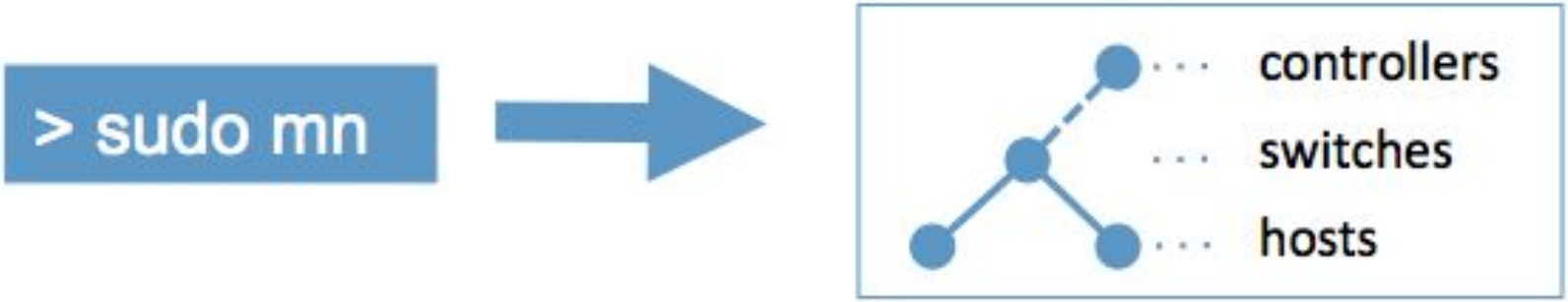
Mininet

Introducción a los Sistemas Distribuidos (75.43)

Universidad de Buenos Aires, Facultad de Ingeniería

Mininet



Crea una red virtual, ejecutando kernel, switch y código de aplicación, en una sola máquina.
Se utiliza para probar, desarrollar, y experimentar con OpenFlow y SDN.(1)

(1) http://mininet.org/

Mininet - Comandos

- **Lanzar mininet (CLI):**
  $ sudo mn
- **Listar nodos:**
  mininet> nodes
- **Listar enlaces:**
  mininet> links
- **Listar información de los nodos:**
  mininet> dump
- **Ejecutar comando en un device:**
  mininet> dev1 command
  mininet> h1 ifconfig -a
- **Probar conectividad entre hosts:**
  mininet> h1 ping h2
- **Abrir una xterm:**
  mininet> h1 xterm
- **Ping a todos:**
  mininet> pingall

Mininet - Ejemplo

Lanzar una instancia de mininet, sin parámetros. Analizar la topología y probar la conectividad entre dispositivos.

$ sudo mn

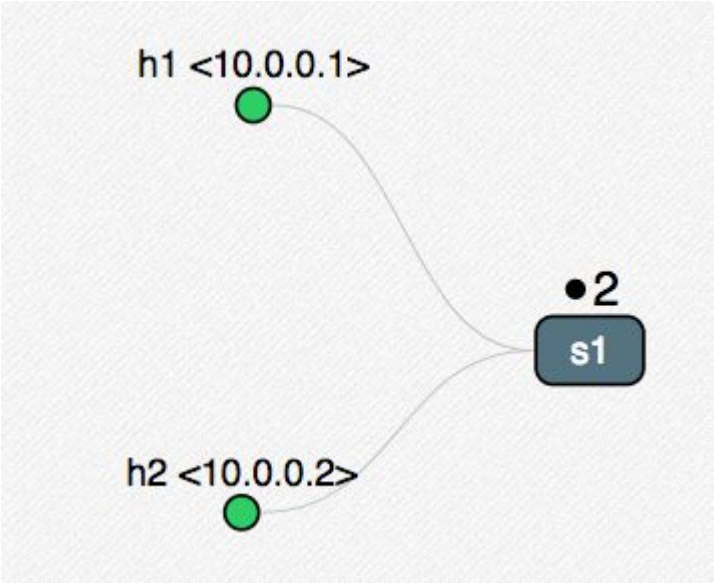
Por defecto, se crea una topología con un switch y dos hosts.

```
➜  ~ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Mininet - Ejemplo

mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)

mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=1907>
<Host h2: h2-eth0:10.0.0.2 pid=1909>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=1914>
<Controller c0: 127.0.0.1:6653 pid=1900>



Mininet - Ejemplo

Probar la conectividad entre los dispositivos

mininet> pingall
***** Ping: testing ping reachability**
h1 -> h2
h2 -> h1
***** Results: 0% dropped (2/2 received)**

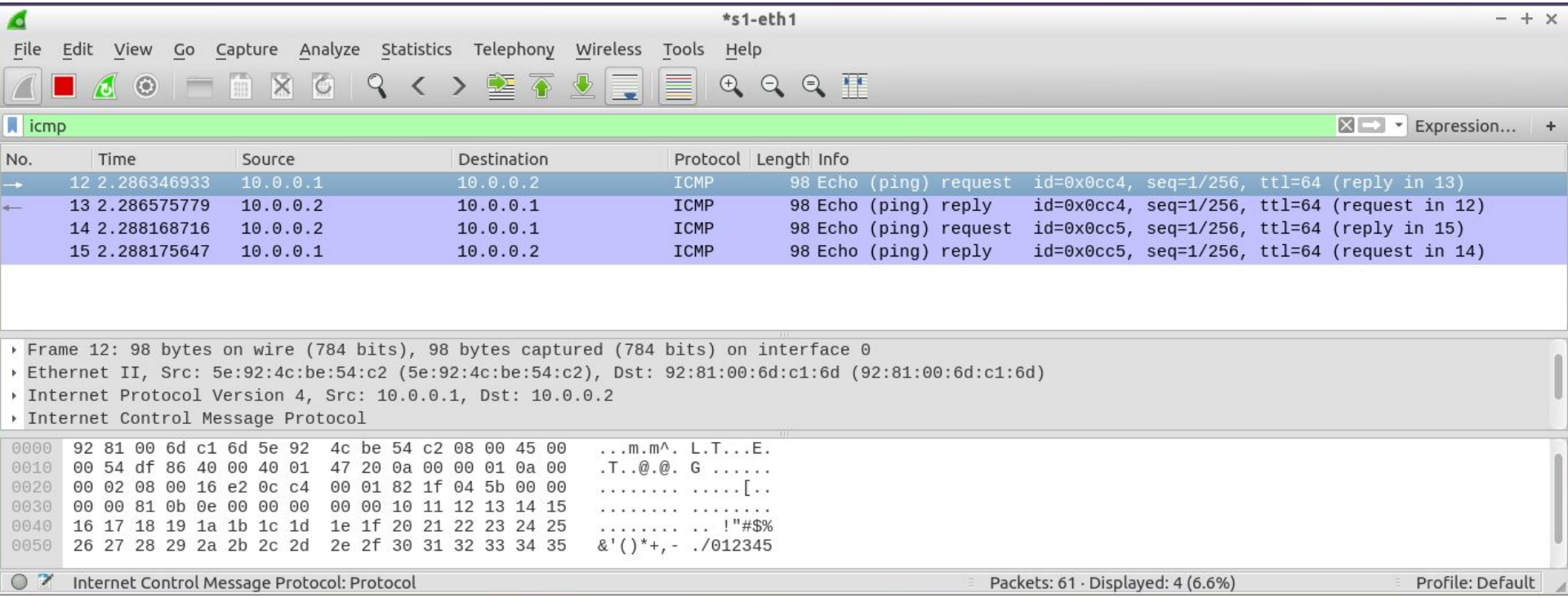
El comando pingall se utiliza para probar la conectividad entre todos los dispositivos, se ejecuta el comando ping en todos los dispositivos de la topología

> dev1 ping -c1 dev2

Otra forma

➜ **~ sudo mn --test pingall**

Mininet - Ejemplo



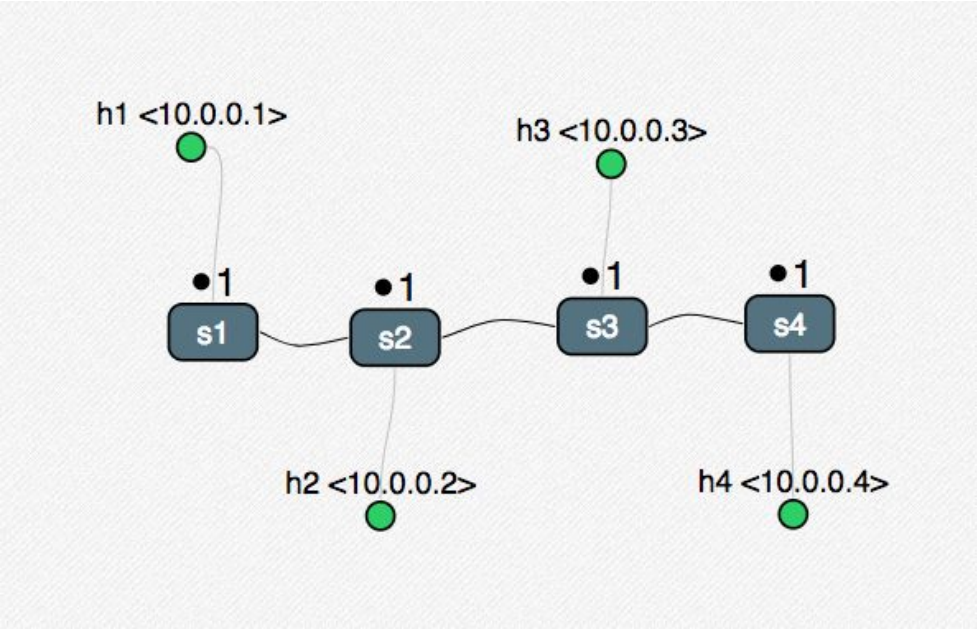
Mininet - Topologías más complejas

```
➜  ~ sudo mn --topo linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>
```

Topología "linear", recibe por parámetro la cantidad de switches, y crea una topología donde cada switch tiene un host conectado, y todos los switches están conectados en forma lineal.

Mininet - Topologías más complejas

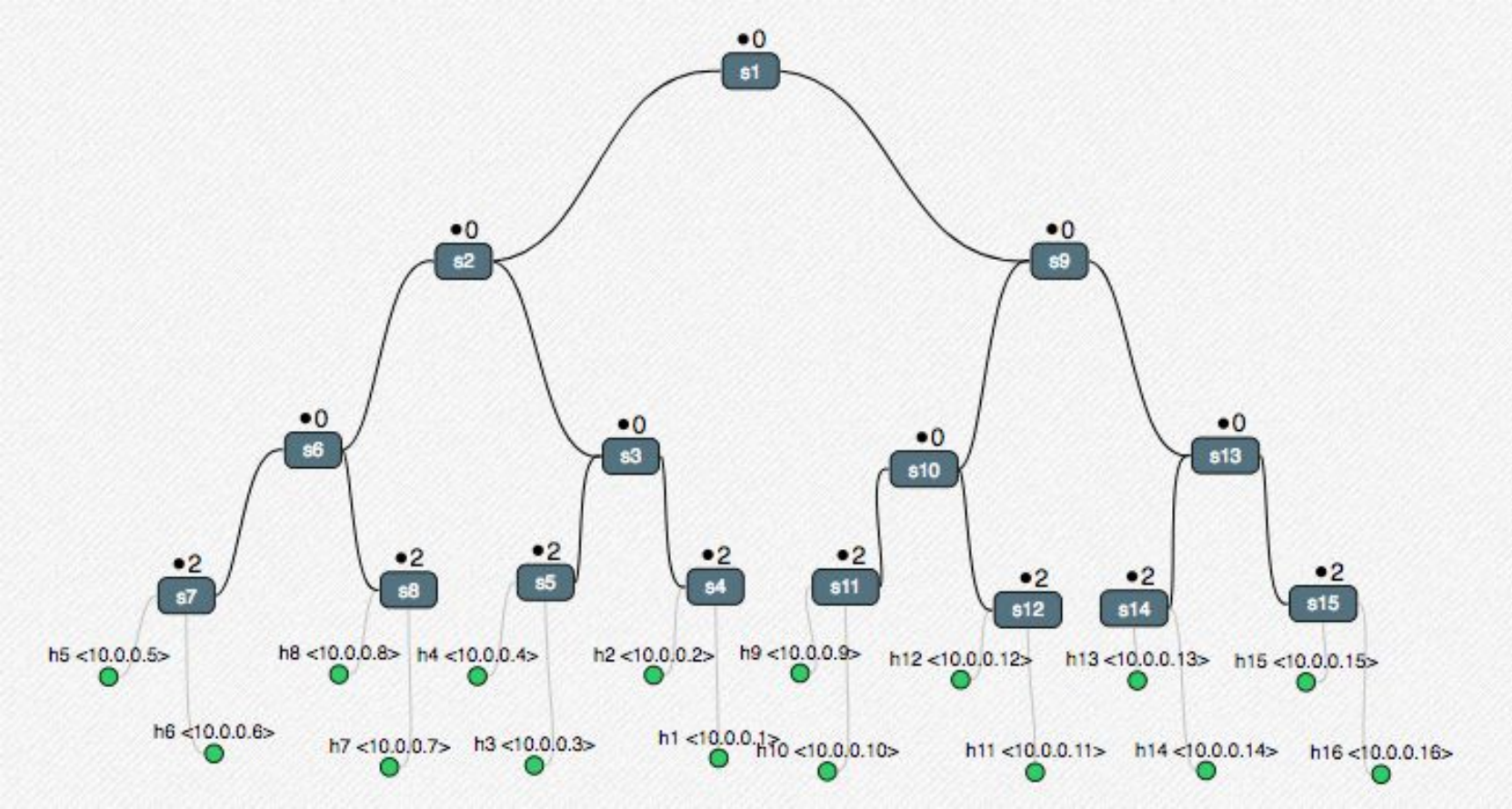
➜ ~ sudo mn --topo linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>



Mininet - Topologías más complejas

```
➜  ~ sudo mn --topo tree,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12
h13 h14 h15 h16
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14
s15
*** Adding links:
(s1, s2) (s1, s9) (s2, s3) (s2, s6) (s3, s4) (s3,
s5) (s4, h1) (s4, h2) (s5, h3) (s5, h4) (s6, s7)
(s6, s8) (s7, h5) (s7, h6) (s8, h7) (s8, h8) (s9,
s10) (s9, s13) (s10, s11) (s10, s12) (s11, h9)
(s11, h10) (s12, h11) (s12, h12) (s13, s14)
(s13, s15) (s14, h13) (s14, h14) (s15, h15)
(s15, h16)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12
h13 h14 h15 h16
*** Starting controller
c0
*** Starting 15 switches
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13
s14 s15 ...
*** Starting CLI:
mininet>
```

Mininet - Topologías más complejas



Mininet - Topologías personalizadas

Mininet provee las siguientes topologías:

- linear
- minimal
- reversed
- single
- torus
- tree

Sin embargo, también se pueden crear topologías personalizadas usando la API Python que nos provee la herramienta.

Mininet - Topologías personalizadas

```
from mininet.topo import Topo

class MyTopo( Topo ):
    "Simple topology example."

    def __init__( self ):
        "Create custom topo."

        # Initialize topology
        Topo.__init__( self )

        # Add hosts and switches
        leftHost = self.addHost( 'h1' )
        rightHost = self.addHost( 'h2' )
        leftSwitch = self.addSwitch( 's3' )
        rightSwitch = self.addSwitch( 's4' )

        # Add links
        self.addLink( leftHost, leftSwitch )
        self.addLink( leftSwitch, rightSwitch )
        self.addLink( rightSwitch, rightHost )


topos = { 'mytopo': ( lambda: MyTopo() ) }
```

custom/topo-2sw-2host.py

Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```

--custom file.py: Ubicación del archivo donde está definida la topología

Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch
ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```

--topo nombre: Nombre de la topología

topos = { example: (lambda: ExampleTopology()) }

Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac
--switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```

--mac:
Se configuran direcciones MAC sencillas para un usuario

Sin --mac:
HWaddr 9a:e9:bf:6c:86:95

Con --mac:
HWaddr 00:00:00:00:00:01

Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```

--switch ovsk:
Se utiliza un tipo de switch
"**Open vSwitch**"(2)

(2) http://www.openvswitch.org/

Mininet

- ¿Es posible levantar un servidor?
- ¿Cómo hago que un host le haga un request a otro host/servidor?

Mininet - iperf

➜ **~ man iperf**

Es una herramienta para realizar mediciones de rendimiento de la red. Puede probar el rendimiento TCP o UDP. Para realizar una prueba iperf, el usuario debe establecer tanto un servidor (para descartar tráfico) como un cliente (para generar tráfico).

Mininet - iperf

Modo servidor:

➜ ~ sudo iperf -s -p 80

Server listening on TCP port 80
TCP window size: 85.3 KByte (default)

Modo cliente:

➜ ~ sudo iperf -c 10.0.0.1 -p 80

Client connecting to localhost, TCP port 80
TCP window size: 2.50 MByte (default)

---

[3] local 127.0.0.1 port 48512 connected with 127.0.0.1 port 80
[ID] Interval Transfer Bandwidth
[3] 0.0-10.0 sec 56.9 GBytes 48.9 Gbits/sec

Mininet - iperf

Desde la consola de Mininet

mininet> iperf h1 h2
***** Iperf: testing TCP bandwidth between h1 and h2**
***** Results: ['24.9 Gbits/sec', '25.0 Gbits/sec']**